

Facial Age Estimation from Images using Image processing Techniques and Neural Networks

*An Industry Oriented Mini Project Report submitted to
Jawaharlal Nehru Technological University, Hyderabad
in partial fulfillment of the requirements for the award of Degree of*

BACHELOR OF TECHNOLOGY

in

Computer Science and Engineering (Cyber Security)

Submitted by

PANDUGA PAVAN

Roll No:22831A6244

KOMPALLY AKSHAY

Roll No:22831A6217

BHEEMANAVIONA SHASHIKIRAN

Roll No:22831A6206

Under the Guidance of
Mrs. RUPINDER SAINI
Assistant Professor



**Department of Computer Science and Engineering
(Cyber Security)**

**Guru Nanak Institute of Technology
Ibrahimpattanam, Hyderabad, R.R. District – 501506**

May 2025

CERTIFICATE

This is to certify that the Industry Oriented Mini-project “**Facial Age Estimation from Images using Image processing Techniques and Neural Networks**” is being submitted by **Mr. PANDUGA PAVAN** bearing Roll No: **22831A6244**, **Mr. KOMPALLY AKSHAY** bearing Roll No: **22831A16217** and **Mr. BHEEMANAVIONA SHASHI KIRAN** bearing Roll No: **22831A6206** in partial fulfillment for the award of the Degree of Bachelor of the Technology in **Computer Science & Engineering (Cyber Security)** to the Jawaharlal Nehru Technological University is a record of Bonafide work carried out by them under my guidance and supervision.

The results embodied in this Mini-project report have not been submitted to any other University or Institute for the award of any Degree or Diploma.

Internal Guide

Mrs. RUPINDER SAINI

Assistant Professor

Project Co-Ordinator

Mr. E. VINAY KUMAR

Assistant Professor

Head of the Department

Dr. CH. BALAKRISHNA

External Examiner

Declaration Of Student

We, **Mr. PANDUGA PAVAN (22831A6244), Mr. KOMPALLY AKSHAY (2283A16217), Mr.BHEEMANAVIONA SHASHIKIRAN (22831A6206)**, hereby declare that the Industry Oriented Minor Project titled “**Facial Age Estimation from Images using Image processing Techniques and Neural Networks**” has been carried out by us as part of the requirements for the award of the Degree of **Bachelor of Technology** in the Department of **Computer Science & Engineering (Cyber Security)** at Guru Nanak Institute of Technology.

We confirm the following:

1. The project was undertaken by us under the supervision of our guide, **Mrs. Rupinder Saini**, from the selection of the topic to the completion of the final report.
2. I have ensured that the results presented in the report are accurate and based on our original work.
3. To the best of our knowledge, the content of this report is free from plagiarism and adheres to ethical standards.
4. Each member of the team has contributed significantly and appropriately to the Industry Oriented Mini Project work.
5. The project report has been prepared with diligence, ensuring clarity, accuracy, and adherence to academic standards.

We further declare that this report has not been submitted, in part or full, to any other institution or university for the award of any degree or diploma.

PANDUGA PAVAN

Signature

22831A2644

KOMPALLY AKSHAY

Signature

22831A2617

BHEEMANAVIONA SHASHIKIRAN

Signature

22831A6206

Date:

Place: Ibrahimpatnam

Declaration of Guide

I, **Mrs. Rupinder Saini**, hereby declare that I have guided the Industry Oriented Mini Project titled “**Facial Age Estimation From Images Using Image Processing Techniques and Neural Networks**” undertaken by **Mr. Panduga Pavan (22831A6244)**, **Mr. Kompally Akshay (22831A6217)**, **Mr. Bheemanaviona Shashikiran (22831A6206)**. This project was carried out towards the fulfillment of the requirements for the award of the Degree of **Bachelor of Technology in Computer Science & Engineering (Cyber Security)** at Guru Nanak Institute of Technology. As the guide, I confirm the following:

1. I have overseen the entire project process, from the selection of the project title to the submission of the final report.
2. I have reviewed and certified the accuracy and relevance of the results presented in the report.
3. The work carried out is original, free from plagiarism, and adheres to ethical guidelines.
4. The contributions of each student have been appropriately recognized and assessed.
5. The project report has been prepared under my supervision, ensuring adherence to high standards of quality, clarity, and structure.

I further certify that this project report has not been previously submitted in part or full for the award of any degree or diploma by any institution or university.

Name of Guide: Mrs. Rupinder Saini

Signature of the Guide

Date:

Place: Ibrahimpatnam

Name of HOD: Dr. CH. Balakrishna

Signature of the HOD

Department: Computer Science & Engineering (Cyber Security)

Date:

Place: Ibrahimpatnam

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our internal guide, **Mrs. Rupinder Saini**, Assistant Professor, **Computer Science & Engineering (Cyber Security)**, for his valuable guidance, encouragement, and continuous support throughout the duration of this Industry Oriented Mini Project.

We are also thankful to **Dr.CH. Balakrishna**, Head of the Department, **Computer Science & Engineering (Cyber Security)**, for his expert supervision and helpful suggestions, which contributed significantly to the successful completion of this Industry Oriented Mini Project.

We would also like to thank the faculty members of the **Computer Science and Engineering (Cyber Security)**, and the Lab Technicians for their assistance and cooperation during the practical work of our Industry Oriented Mini Project.

We are grateful to our friends and well-wishers for their encouragement, collaboration, and useful feedback throughout the Industry Oriented Mini Project journey.

Lastly, we sincerely thank our parents for their constant support, patience, and motivation, which helped us complete this Industry Oriented Mini Project successfully.

PANDUGA PAVAN

22831A2644

KOMPALLY AKSHAY

22831A6217

BHEEMANAVIONA SHASHIKIRAN

22831A6206

TABLE OF CONTENTS

DESCRIPTION	PAGE NUMBER
CERTIFICATE	ii
DECLARATION OF STUDENT	iii
DECLARATION OF GUIDE	iv
ACKNOWLEDGEMENT	v
ABSTRACT	ix
LIST OF FIGURES	x
LIST OF TABLES	xii
1. INTRODUCTION	1
1.1 GENERAL	1
1.2 OBJECTIVE	2
1.3 EXISTING SYSTEM	3
1.3.1 EXISTING SYSTEM DISADVANTAGES	3
1.4 PROPOSED SYSTEM	3
1.4.1 PROPOSED SYSTEM ADVANTAGES	3
2. LITERATURE SURVEY	4
2.1 LITERATURE SURVEY	4
3. DESIGN AND DEVELOPMENT	9
3.1 PROBLEM STATEMENT	9
3.2 SYSTEM REQUIREMENTS	9
3.2.1 GENERAL	9
3.2.2 HARDWARE REQUIREMENTS	9
3.2.3 SOFTWARE REQUIREMENTS	10
3.2.4 FUNCTIONAL REQUIREMENTS	10
3.2.5 NON-FUNCTIONAL REQUIREMENTS	11
3.3 SYSTEM ARCHITECTURE	12

3.4 DESIGN AND METHODOLOGY	13
3.4.1 GENERAL	13
3.4.2 UML DIAGRAM	14
3.4.2.1 USE CASE DIAGRAM	14
3.4.2.2 CLASS DIAGRAM	15
3.4.2.3 OBJECT DIAGRAM	16
3.4.2.4 STATE DIAGRAM	17
3.4.2.5 ACTIVITY DIAGRAM	18
3.4.2.6 SEQUENCE DIAGRAM	19
3.4.2.7 COLLABORATION DIAGRAM	20
3.4.2.8 COMPONENT DIAGRAM	21
3.4.2.9 DEPLOYMENT DIAGRAM	22
3.4.2.10 DATA FLOW DIAGRAM	23
3.5 TOOLS AND TECHNOLOGIES	25
3.5.1 PYTHON	25
3.5.2 HISTORY OF PYTHON	25
3.5.3 IMPORTANCE OF PYTHON	25
3.5.4 FEATURES OF PYTHON	26
3.5.5 LIBRARIES USED IN PYTHON	27
3.6 TECHNIQUES OR ALGORITHMS USED	28
3.7 IMPLEMENTATION	29
3.8 TESTING AND VALIDATION	36
3.8.1 GENERAL	36
3.8.2 DEVELOPING METHODOLOGIES	36
3.8.3 TYPES OF TESTING	36
3.8.3.1 UNIT TESTING	36
3.8.3.2 FUNCTIONAL TESTING	37
3.8.3.3 SYSTEM TESTING	37
3.8.3.4 PERFORMANCE TESTING	37
3.2.3.5 INTEGRATION TESTING	37

3.8.3.6 ACCEPTANCE TESTING	38
3.8.3.7 BUILT THE TEST PLAN	38
3.9 SNAPSHOTS	39
3.9.1 GENERAL	39
3.9.2 VARIOUS SNAPSHOTS	39
4. RESULT AND DISCUSSION	44
4.1 RESULT	44
4.2 FUTURE ENHANCEMENT	44
5. CONCLUSION	45
5.1 CONCLUSION	45
5.2 REFERENCES	46

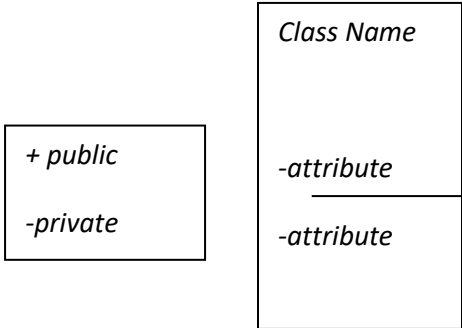
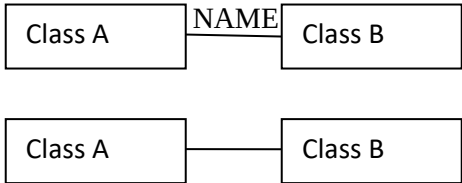
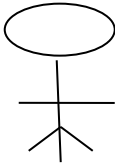
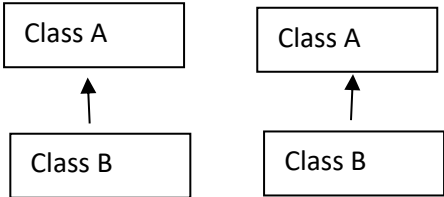
ABSTRACT

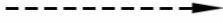
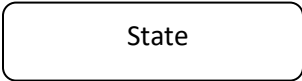
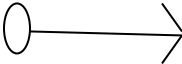
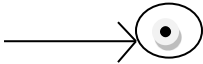
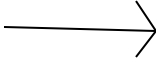
It has been recently shown that Generative Adversarial Networks (GANs) can produce synthetic images of exceptional visual fidelity. In this work, we propose the first GAN based method for automatic face aging. Contrary to previous works employing GANs for altering of facial attributes, we make a particular emphasize on preserving the original person's identity in the aged version of his/her face. To this end, we introduce a novel approach for "Identity-Preserving" optimization of GAN's latent vectors. The objective evaluation of the resulting aged and rejuvenated face images by the state-of-the-art face recognition and age estimation solutions demonstrates the high potential of the proposed method.

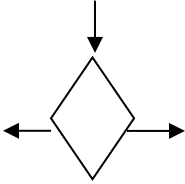
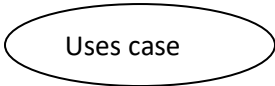
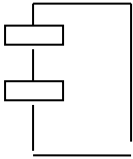
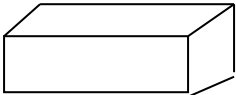
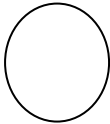
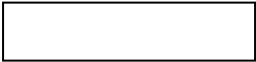
LIST OF FIGURES

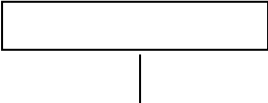
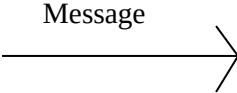
FIGURE NO	NAME OF THE FIGURE	PAGE NO
3.4.2.1	Use Case Diagram	14
3.4.2.2	Class Diagram	15
3.4.2.3	Object Diagram	16
3.4.2.4	State Diagram	17
3.4.2.5	Activity Diagram	18
3.4.2.6	Sequence Diagram	19
3.4.2.7	Collaboration Diagram	20
3.4.2.8	Component Diagram	21
3.4.2.9	Deployment Diagram	22
3.4.2.10	Data Flow Diagram	23

LIST OF SYMBOLS

S.NO	NOTATION NAME	NOTATION	DESCRIPTION
1.	Class		Represents a collection of similar entities grouped together.
2.	Association		Associations represents static relationships between classes. Roles represents the way the two classes see each other.
3.	Actor		It aggregates several classes into a single classes.
4.	Aggregation		Interaction between the system and external environment

5.	Relation (uses)	Uses 	Used for additional process communication.
6.	Relation (extends)		Extends relationship is used when one use case is similar to another use case but does a bit more.
7.	Communication		Communication between various use cases.
8.	State		State of the processes.
9.	Initial State		Initial state of the object
10.	Final state		Final state of the object
11.	Control flow		Represents various control flow between the states.

12.	Decision box		Represents decision making process from a constraint
13.	Use case		Interact ion between the system and external environment.
14.	Component		Represents physical modules which are a collection of components.
15.	Node		Represents physical modules which are a collection of components.
16.	Data Process/State		A circle in DFD represents a state or process which has been triggered due to some event or action.
17.	External entity		Represents external entities such as keyboard,sensors,etc.

18.	Transition		Represents communication that occurs between processes.
19.	Object Lifeline		Represents the vertical dimensions that the object communications.
20.	Message		Represents the message exchanged.

CHAPTER 1

INTRODUCTION

1.1 GENERAL

Face aging, also known as age synthesis and age progression, is defined as aesthetically rendering a face image with natural aging and rejuvenating effects on the individual face. It has plenty of applications in various domains including cross-age face recognition, finding lost children and entertainment. Traditional face aging approaches can be roughly split between the prototyping ones and the modeling one. Prototyping approaches estimate average faces within predefined age groups. The differences between these faces constitute the aging patterns which are further used to transform an input face image into the target age group. The prototype approaches are simple and fast, but since they are based on general rules, they totally discard the personalized information which results in unrealistic images. On the contrary, modeling approaches employ parametric models to simulate the aging mechanisms of muscles, skin and skull of a particular individual. However, these approaches often require face aging sequences of the same person with wide range of ages which are very costly to collect.

The presented traditional face aging approaches are limited to modeling of the aging patterns missing the global comprehension of a human face (its personality traits, facial expression, possible facial accessories etc.) However, in many real-life use cases, face aging must be combined with otherface alterations, such as adding sunglasses or beard. These non-trivial modifications require global generative models of human faces. Natural image generation has been thoroughly studied for years, but it has not been until 2014 when advances in deep learning has allowed to produce image samples and interpolations of very high visual fidelity.

1.2 OBJECTIVE

The objective evaluation of the resulting aged and rejuvenated face images by the state-of-the-art face recognition and age estimation solutions demonstrates the high potential of the proposed method.

1.3 EXISTING SYSTEM

The presented traditional face aging approaches are limited to modeling of the aging patterns missing the global comprehension of a human face (its personality traits, facial expression, possible facial accessories etc.) However, in many real-life use cases, face aging must be combined with other face alterations, such as adding sunglasses or beard. These non-trivial modifications require global generative models of human faces. Natural image generation has been thoroughly studied for year, when advances in deep learning has allowed to produce image samples and interpolations of very high visual fidelity.

1.3.1 EXISTING SYSTEM DISADVANTAGES

- Keeping records of customers
- Analysing data

1.4 PROPOSED SYSTEM

The key model which has made it possible is called Generative Adversarial Network (GAN). Contrary to auto encoders (for example, variational auto encoders) which optimize the L2 reconstruction loss and hence produce blurry images, GANs are explicitly trained to generate the most plausible and realistic images which are hard to distinguish from real data. GANs have already been employed to perform modifications on human faces, such as changing the hair color, adding sunglasses or even binary aging (i.e., simply making face look older or younger without précising particular age categories).

1.4.1 PROPOSED SYSTEM ADVANTAGES

- Easy to analysis

CHAPTER 2

LITERATURE SURVEY

2.1 LITERATURE SURVEY:

Title:Age synthesis and estimation via faces: A survey

Year:2010

Authors:Yun Fu, Guodong Guo, and Thomas S Huang

Description:Human age, as an important personal trait, can be directly inferred by distinct patterns emerging from the facial appearance. Derived from rapid advances in computer graphics and machine vision, computer-based age synthesis and estimation via faces have become particularly prevalent topics recently because of their explosively emerging real-world applications, such as forensic art, electronic customer relationship management, security control and surveillance monitoring, biometrics, entertainment, and cosmetology. Age synthesis is defined to retender a face image aesthetically with natural aging and rejuvenating effects on the individual face. Age estimation is defined to label a face image automatically with the exact age (year) or the age group (year range) of the individual face. Because of their particularity and complexity, both problems are attractive yet challenging to computer-based application system designers. Large efforts from both academia and industry have been devoted in the last a few decades. In this paper, we survey the complete state-of-the-art techniques in the face image-based age synthesis and estimation topics. Existing models, popular algorithms, system performances, technical difficulties, popular face aging databases, evaluation protocols, and promising future directions are also provided with systematic discussions.

Title:Personalized age progression with aging dictionary

Year:2015

Author:iangbo Shu, Jinhui Tang, Hanjiang Lai, Luoqi Liu, and Shuicheng Yan

Description:In this paper, we aim to automatically render aging faces in a personalized way. Basically, a set of age-group specific dictionaries are learned, where the dictionary bases corresponding to the same index yet from different dictionaries form a particular aging process pattern cross different age groups, and a linear combination of these patterns expresses a particular personalized aging process. Moreover, two factors are taken into consideration in the dictionary learning process. First, beyond the aging dictionaries, each subject may have extra personalized facial characteristics, e.g. mole, which are invariant in the aging process. Second, it is challenging or even impossible to collect faces of all age groups for a particular subject, yet much easier and more practical to get face pairs from neighboring age groups. Thus a personality-aware coupled reconstruction loss is utilized to learn the dictionaries based on face pairs from neighboring age groups. Extensive experiments well demonstrate the advantages of our proposed solution over other state-of-the-arts in term of personalized aging progression, as well as the performance gain for cross-age face verification by synthesizing aging faces.

Title: Age-invariant face recognition

Year:2010

Author:X. W. Chen and X. Lin.

Description:Age-invariant face recognition is one of the most crucial computer vision problems, e.g., in passport verification, surveillance systems, and missing individuals identification. The extraction of robust face features is a challenge since the facial characteristics change over age progression. In this paper, an age-invariant face recognition system is proposed, which includes four stages: preprocessing, feature extraction, feature fusion, and classification. Preprocessing stage detects faces using Viola–Jones algorithm and frontal face alignment. Feature extraction is achieved using a CNN architecture using VGG-Face model to extract compact face features. Extracted features are fused using the real-time feature-level multi-discriminant correlation analysis, which significantly reduces feature dimensions and results in the most relevant features to age-invariant face recognition. Finally, K-nearest neighbor and support vector machine are investigated for classification. Our experiments are performed on two standard face-aging datasets, namely FGNET and MORPH. Rank-1 recognition accuracy of the proposed system is 81.5% on FGNET and 96.5% on MORPH. Experimental results outperform the current state-of-the-art techniques on same data. These preliminary results show the promise of the proposed system for personal identification despite aging process.

Title:Recurrent face aging

Author:Wei Wang, Zhen Cui, Yan Yan, Jiashi Feng, Shuicheng Yan, Xiangbo Shu, and Nicu Sebe

Year:2016

Description:Modeling the aging process of human face is important for cross-age face verification and recognition. In this paper, we introduce a recurrent face aging (RFA) framework based on a recurrent neural network which can identify the ages of people from 0 to 80. Due to the lack of labeled face data of the same person captured in a long range of ages, traditional face aging models usually split the ages into discrete groups and learn a one-step face feature transformation for each pair of adjacent age groups. However, those methods neglect the in-between evolving states between the adjacent age groups and the synthesized faces often suffer from severe ghosting artifacts. Since human face aging is a smooth progression, it is more appropriate to age the face by going through smooth transition states. In this way, the ghosting artifacts can be effectively eliminated and the intermediate aged faces between two discrete age groups can also be obtained. Towards this target, we employ a twolayer gated recurrent unit as the basic recurrent module whose bottom layer encodes a young face to a latent representation and the top layer decodes the representation to a corresponding older face. The experimental results demonstrate our proposed RFA provides better aging faces over other state-of-the-art age progression methods.

Title:Prototyping and transforming facial textures for perception research

Author:Bernard Tiddeman, Michael Burt, and David Perrett

Year:2001

Description:Transforming facial images along perceived dimensions (such as age, gender, race, or health) has application in areas as diverse as psychology, medicine, and forensics. We can use prototype images to define the salient features of a particular face classification (for example, European female adult or East-Asian male child). We then use the differences between two prototypes to define an axis of transformation, such as younger to older. By applying these changes to a given input face, we can change its apparent age, race, or gender. Psychological investigations reveal a limitation with existing methods that's particularly apparent when changing the age of faces. We relate the problem to the loss of facial textures (such as stubble and wrinkles) in the prototypes due to the blending process. We review the existing face prototyping and transformation methods and present a new, wavelet-based method for prototyping and transforming facial textures

CHAPTER 3

DESIGN AND DEVELOPMENT

3.1 PROBLEM STATEMENT

Accurate estimation of a person's age from facial images is a challenging task due to the wide variability in facial appearance caused by genetics, lifestyle, and environmental factors. Traditional age estimation methods struggle to generalize across diverse populations and lighting conditions. The goal of this project is to design and develop a robust system for Facial Age Estimation using advanced image processing techniques for preprocessing and neural networks for feature extraction and age prediction.

3.2 SYSTEM REQUIREMENTS

3.2.1 GENERAL

These are the requirements for doing the project. Without using these tools and software's we can't do the project. So we have two requirements to do the project. They are

1. Hardware Requirements.
2. Software Requirements.

3.2.2 HARDWARE REQUIREMENTS

The hardware requirements may serve as the basis for a contract for the implementation of the system and should therefore be a complete and consistent specification of the whole system. They are used by software engineers as the starting point for the system design. It should state what the system does and not how it should be implemented.

- PROCESSOR : DUAL CORE 2 DUOS.
- RAM : 4GB DD RAM
- HARD DISK : 250 GB

3.2.3 SOFTWARE REQUIREMENTS

The software requirements document is the specification of the system. It should include both a definition and a specification of requirements. It is a set of what the system should do rather than how it should do it. The software requirements provide a basis for creating the software requirements specification. It is useful in estimating cost, planning team activities, performing tasks and tracking the teams and tracking the team's progress throughout the development activity.

- OPERATING SYSTEM : WINDOWS 7/8/10
- PLATFORM : SPYDER3
- PROGRAMMING LANGUAGE : PYTHON, HTML
- FRONT END : SPYDER3

3.2.4 FUNCTIONAL REQUIREMENTS

A functional requirement defines a function of a software-system or its component. A function is described as a set of inputs, the behavior, This study explores the importance of Customer Segmentation as a core function of CRM as well as the various models for segmenting customers using clustering techniques. The available clustering models for customer segmentation, in general, and the major models of K-Means and Hierarchical Clustering, in particular, are studied and the virtues and vices of the techniques are pointed out.

3.2.5 NON-FUNCTIONAL REQUIREMENTS

The major non-functional Requirements of the system are as follows

➤ **Usability**

The system is designed with completely automated process hence there is no or less user intervention.

➤ **Reliability**

The system is more reliable because of the qualities that are inherited from the chosen platform java. The code built by using java is more reliable.

➤ **Performance**

This system is developing in the high level languages and using the advanced front-end and back-end technologies it will give response to the end user on client system with in very less time.

➤ **Supportability**

The system is designed to be the cross platform supportable. The system is supported on a wide range of hardware and any software platform, which is having JVM, built into the system.

➤ **Implementation**

The system is implemented in web environment using struts framework. The apache tomcat is used as the web server and windows xp professional is used as the platform. Interface the user interface is based on Struts provides HTML Tag.

3.3 SYSTEM ARCHITECTURE:

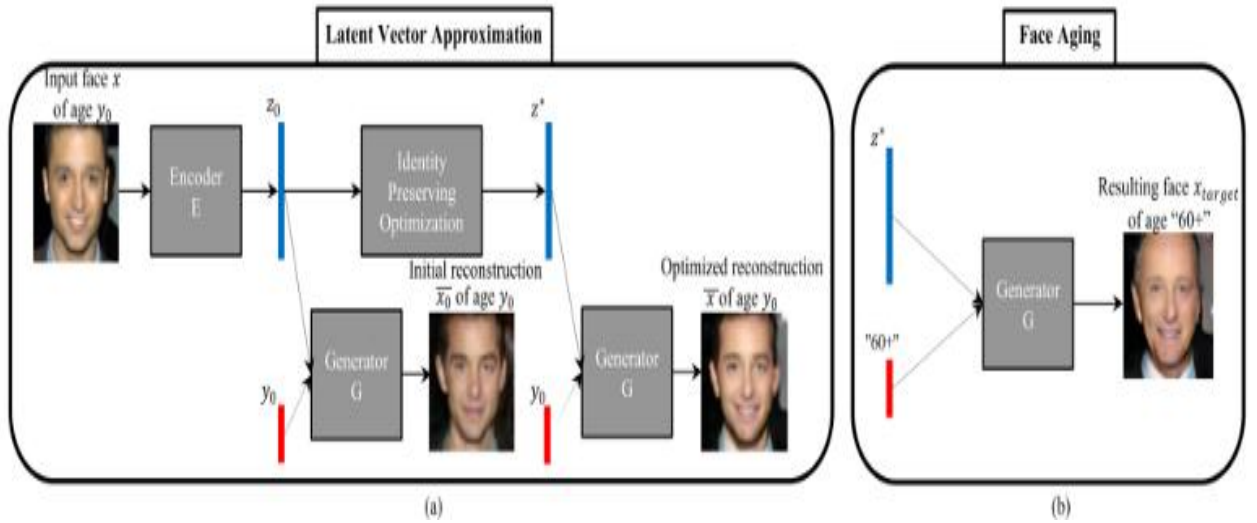


Fig 3.3.1 : System Architecture

3.4 DESIGN AND METHODOLOGY

3.4.1 GENERAL

Design Engineering deals with the various UML [Unified Modelling language] diagrams for the implementation of project. Design is a meaningful engineering representation of a thing that is to be built. Software design is a process through which the requirements are translated into representation of the software. Design is the place where quality is rendered in software engineering. Design is the means to accurately translate customer requirements into finished product.

3.4.2 UML Diagrams

3.4.2.1 USE CASE DIAGRAM:

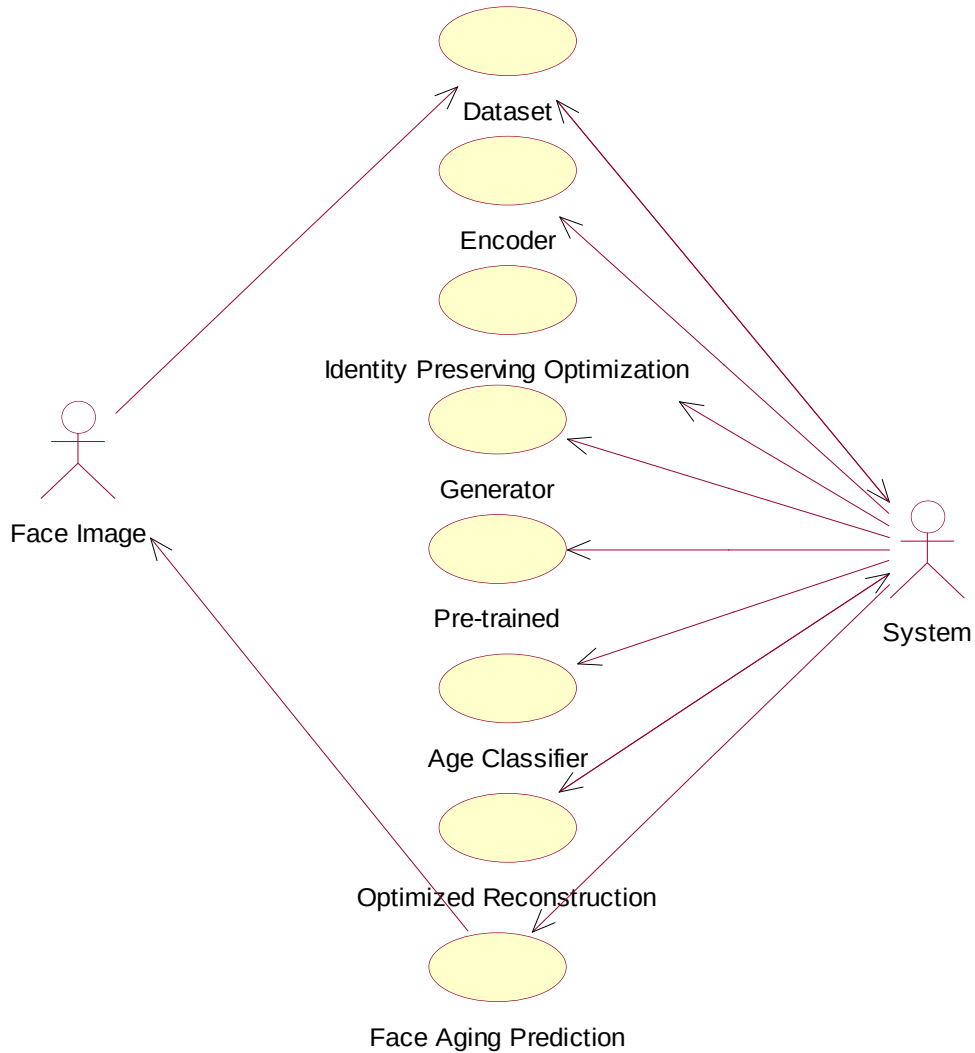


Fig 3.4.2.1 Use Case Diagram

EXPLANATION:

The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted. The above diagram consists of user as actor. Each will play a certain role to achieve the concept.

3.4.2.2 CLASS DIAGRAM:

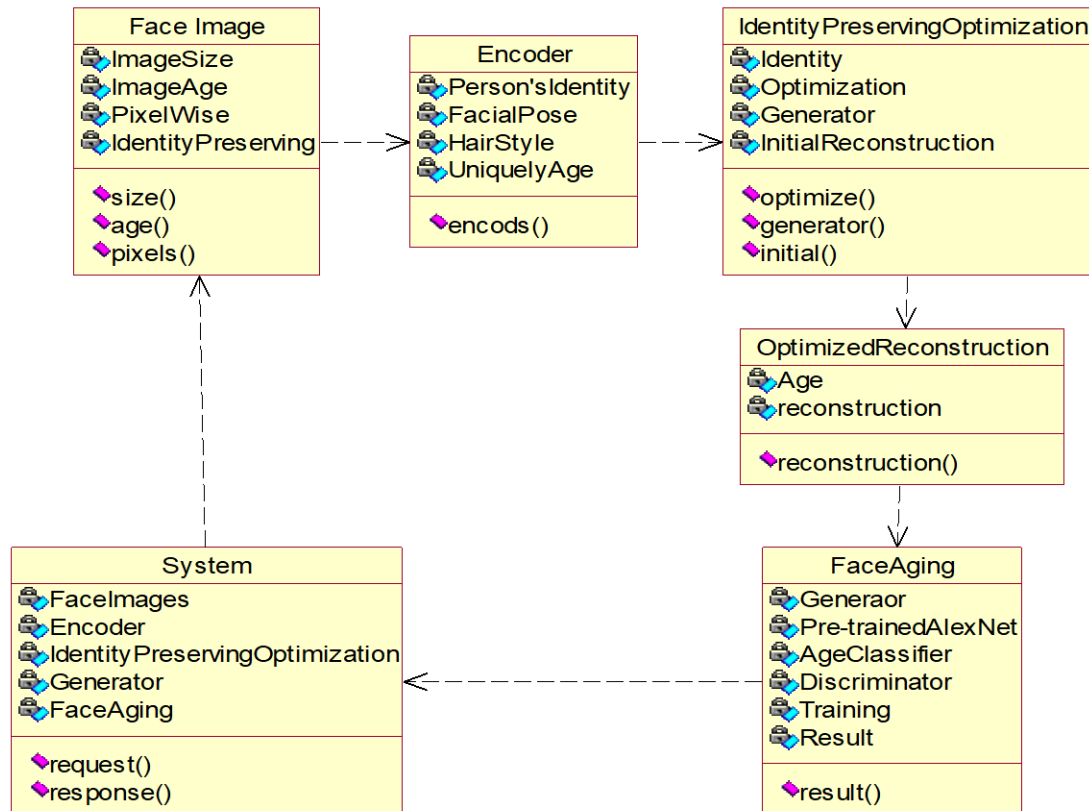


Fig 3.4.4.2 Class Diagram

EXPLANATION:

In this class diagram represents how the classes with attributes and methods are linked together to perform the verification with security. From the above diagram shown the various classes involved in our project.

3.4.2.3 OBJECT DIAGRAM:

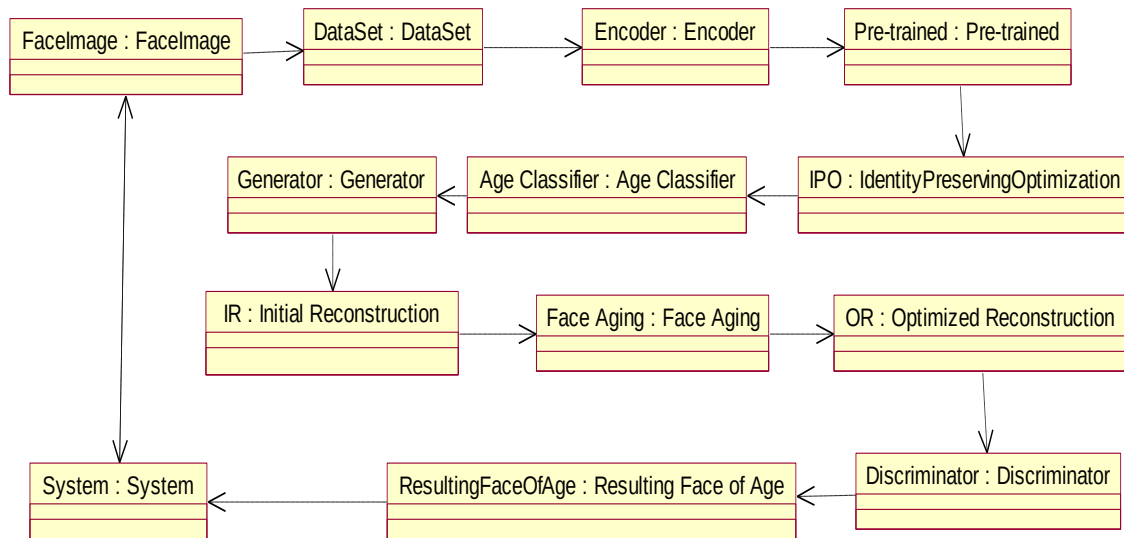


Fig 3.4.2.3: Object Diagram

EXPLANATION:

In the above diagram tells about the flow of objects between the classes. It is a diagram that shows a complete or partial view of the structure of a modeled system. In this object diagram represents how the classes with attributes and methods are linked together to perform the verification with security.

3.4.2.4 STATE DIAGRAM:

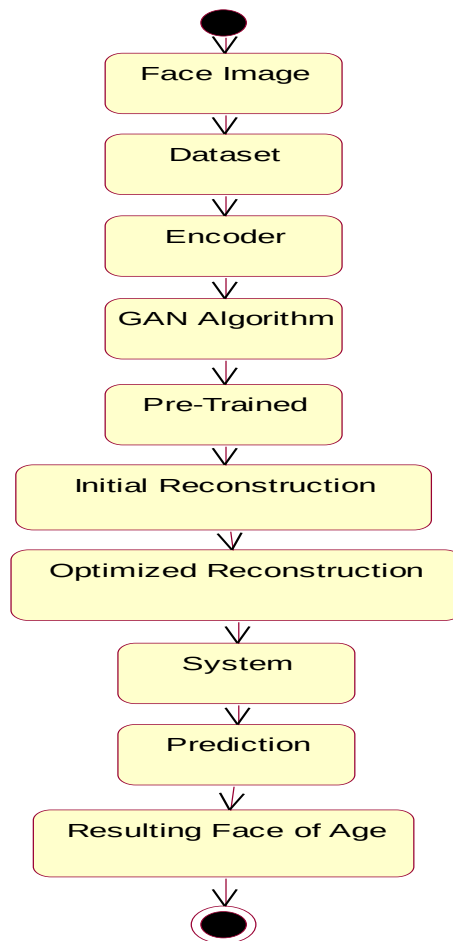


Fig 3.4.2.4 State Diagram

EXPLANATION:

State diagram are a loosely defined diagram to show workflows of stepwise activities and actions, with support for choice, iteration and concurrency. UML, activity diagrams can be used to describe the business and operational step-by-step workflows of components in a system. UML activity diagrams could potentially model the internal logic of a complex operation. In many ways UML activity diagrams are the object-oriented equivalent of flow charts and data flow diagrams (DFDs) from structural development.

3.4.2.5 ACTIVITY DIAGRAM:

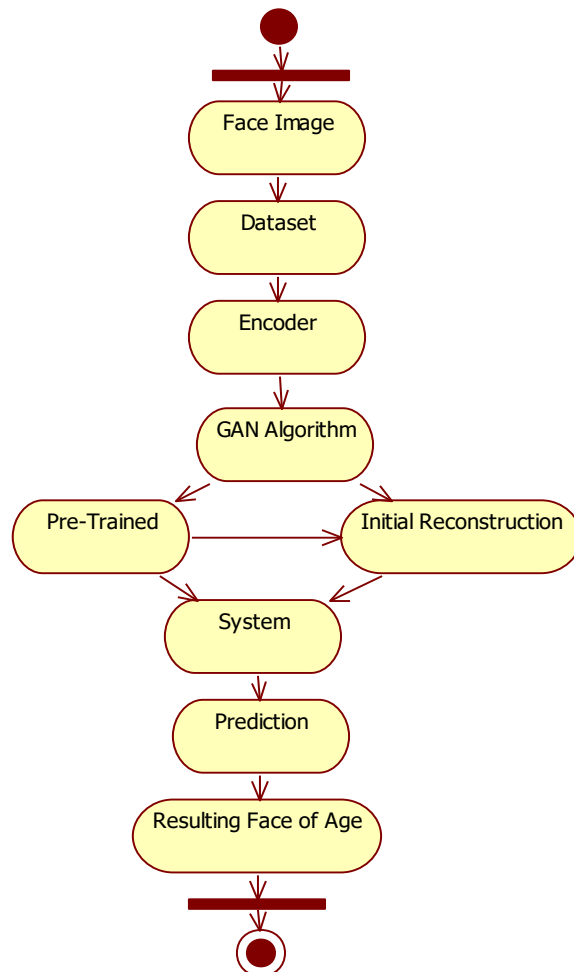


Fig 3.4.2.5 Activity Diagram

EXPLANATION:

Activity diagrams are graphical representations of workflows of stepwise activities and actions with support for choice, iteration and concurrency. In the Unified Modeling Language, activity diagrams can be used to describe the business and operational step-by-step workflows of components in a system. An activity diagram shows the overall flow of control.

3.4.2.6 SEQUENCE DIAGRAM:

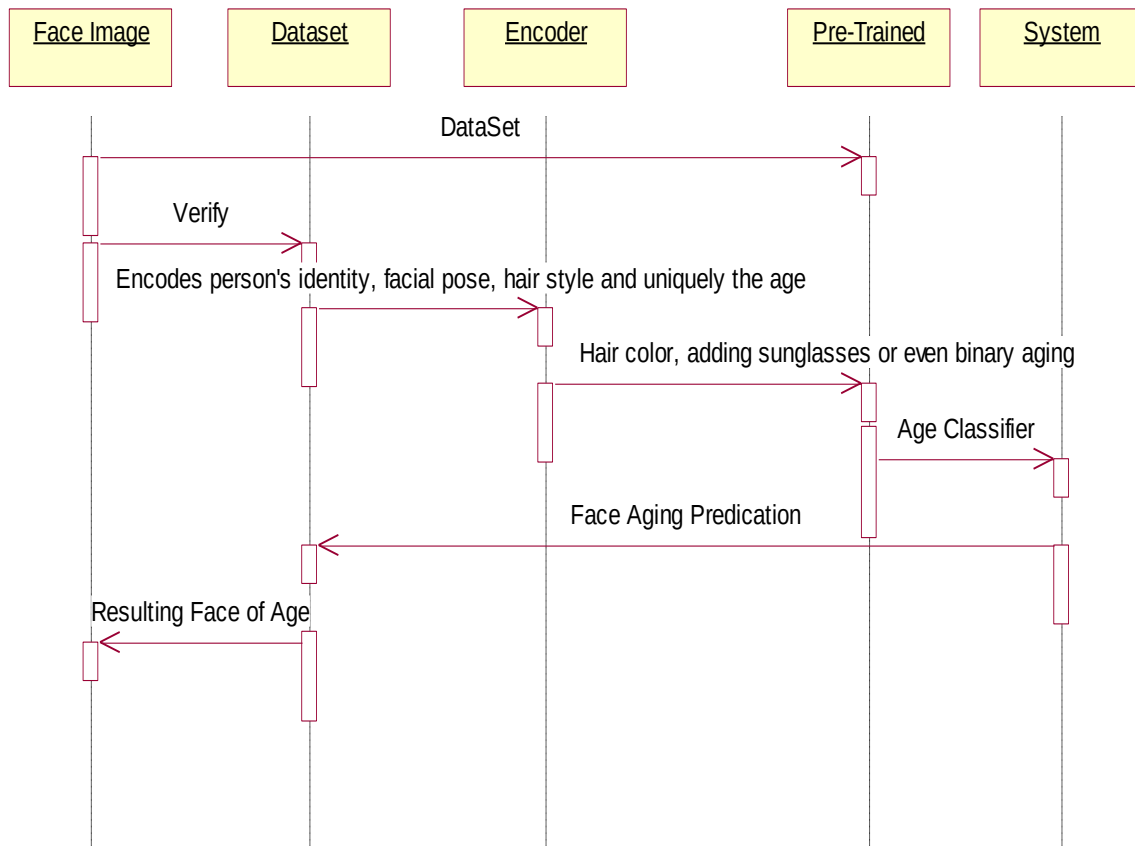


Fig 3.4.2.6: Sequence Diagram

EXPLANATION:

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. A sequence diagram shows object interactions arranged in time sequence. It depicts the objects and classes involved in the scenario and the sequence of messages exchanged between the objects needed to carry out the functionality of the scenario.

3.4.2.7 COLLABORATION DIAGRAM:

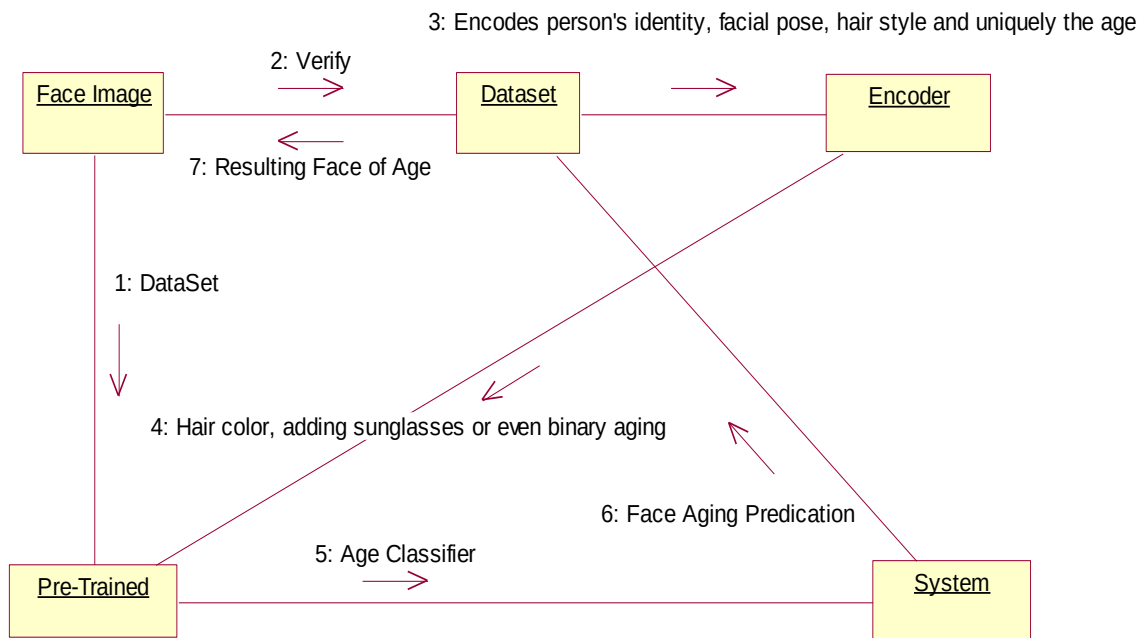


Fig 3.4.2.7 : Collaboration Diagram

EXPLANATION:

A collaboration diagram, also called a communication diagram or interaction diagram, is an illustration of the relationships and interactions among software objects in the Unified Modeling Language (UML). The concept is more than a decade old although it has been refined as modeling paradigms have evolved.

3.4.2.8 COMPONENT DIAGRAM:

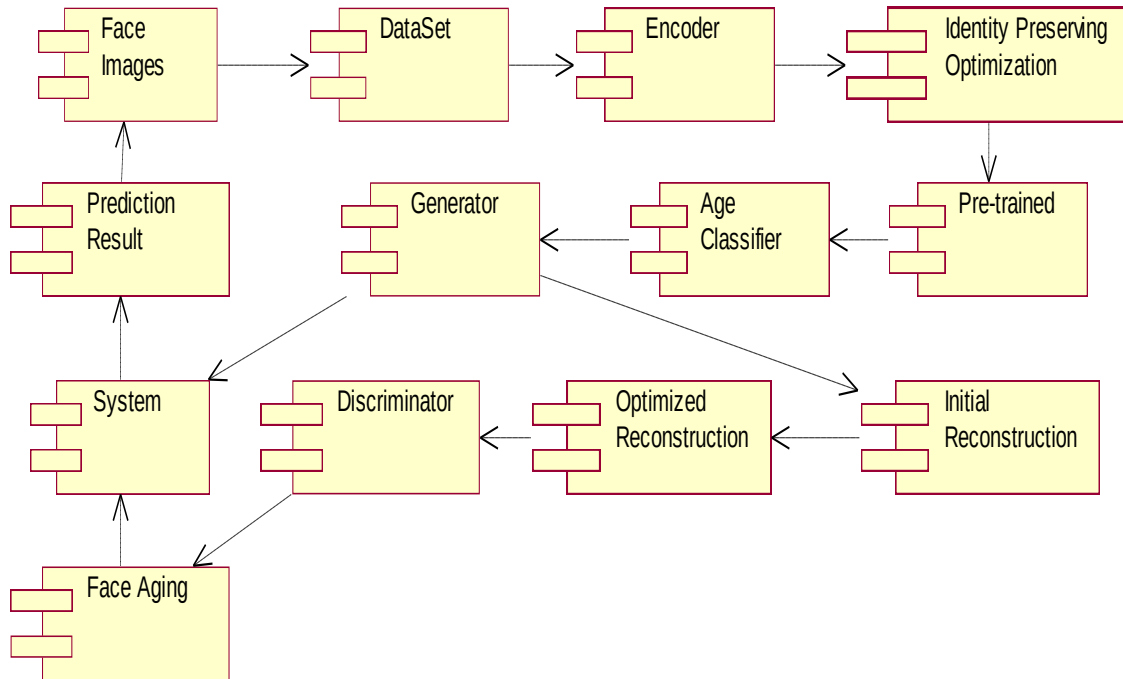


Fig 3.4.2.8 : Component Diagram

EXPLANATION:

In the Unified Modeling Language, a component diagram depicts how components are wired together to form larger components and or software systems. They are used to illustrate the structure of arbitrarily complex systems. User gives main query and it converted into sub queries and sends through data dissemination to data aggregators. Results are to be showed to user by data aggregators. All boxes are components and arrow indicates dependencies.

3.4.2.9 DEPLOYMENT DIAGRAM

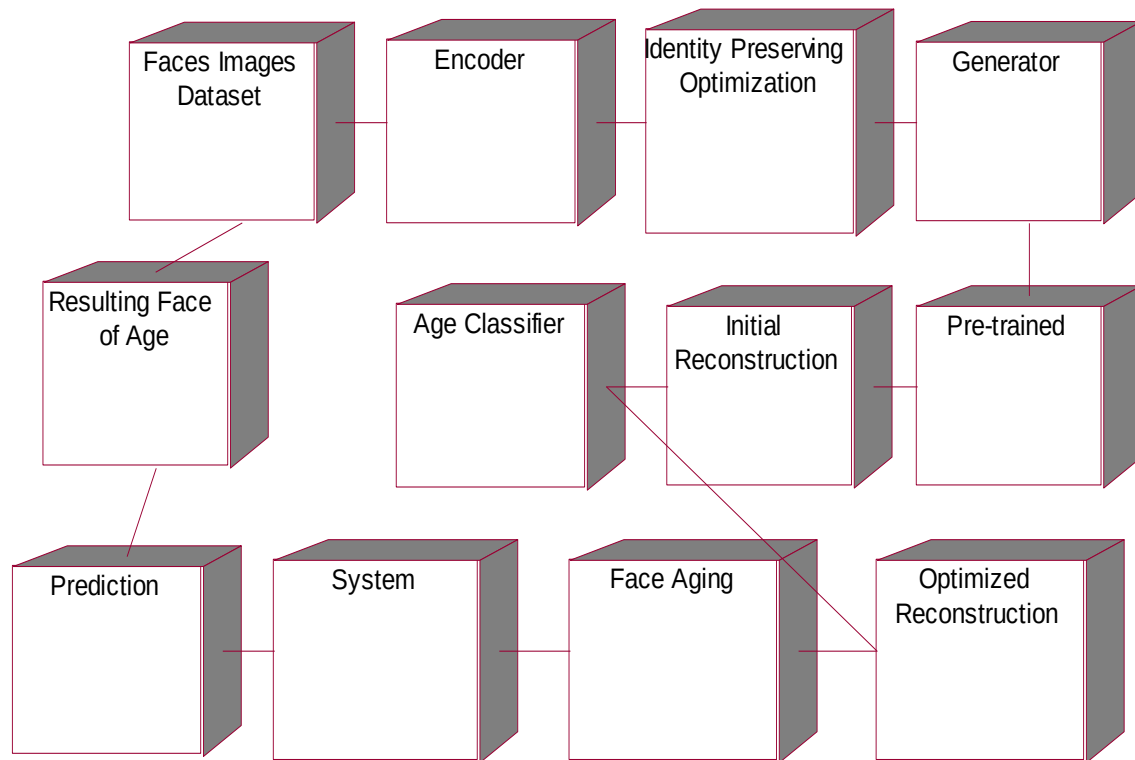


Fig 3.4.2.10 Deployment Diagram

EXPLANATION:

In the Unified Modeling Language, a component diagram depicts how components are wired together to form larger components and or software systems. They are used to illustrate the structure of arbitrarily complex systems. User gives main query and it converted into sub queries and sends through data dissemination to data aggregators. Results are to be showed to user by data aggregators. All boxes are components and arrow indicates dependencies.

3.4.2.10 DATA FLOW DIAGRAM:

Level-0:

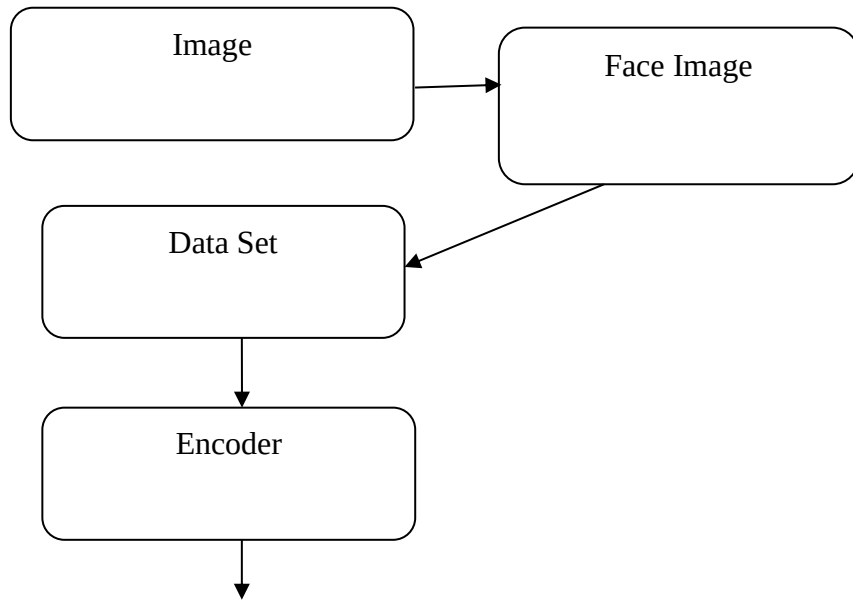


Fig 3.4.2.9 Data Flow Diagram (level 0)

Level-1

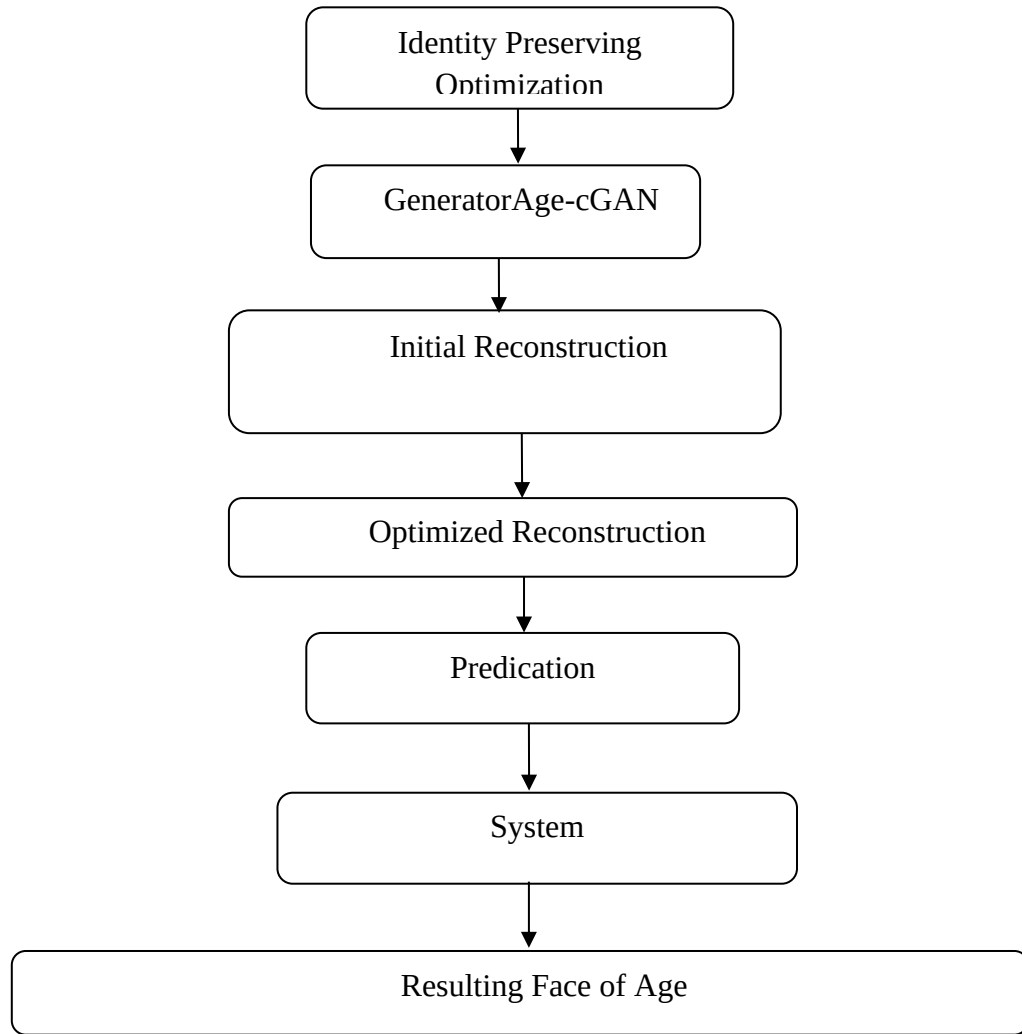


Fig 3.4.2.9: Data Flow Diagram (level 1)

EXPLANATION:

A data flow diagram (DFD) is a graphical representation of the "flow" of data through an information system, modeling its process aspects. Often they are a preliminary step used to create an overview of the system which can later be elaborated. DFDs can also be used for the visualization of data processing (structured design). A DFD shows what kinds of data will be input to and output from the system, where the data will come from and go to, and where the data will be stored. It does not show information about the timing of processes, or information about whether processes will operate in sequence or in parallel.

3.5 TOOLS AND TECHNOLOGIES

3.5.1 Python

Python is a high-level, interpreted, interactive and object-oriented scripting language. Python is designed to be highly readable. It uses English keywords frequently where as other languages use punctuation, and it has fewer syntactical constructions than other languages.

3.5.2 History of Python

Python was developed by Guido van Rossum in the late eighties and early nineties at the National Research Institute for Mathematics and Computer Science in the Netherlands.

Python is derived from many other languages, including ABC, Modula-3, C, C++, Algol-68, SmallTalk, and Unix shell and other scripting languages.

Python is copyrighted. Like Perl, Python source code is now available under the GNU General Public License (GPL).

Python is now maintained by a core development team at the institute, although Guido van Rossum still holds a vital role in directing its progress.

3.5.3 Importance of Python

- **Python is Interpreted** – Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.
- **Python is Interactive** – You can actually sit at a Python prompt and interact with the interpreter directly to write your programs.
- **Python is Object-Oriented** – Python supports Object-Oriented style or technique of programming that encapsulates code within objects.

- **Python is a Beginner's Language** – Python is a great language for the beginner-level programmers and supports the development of a wide range of applications from simple text processing to WWW browsers to games.

3.5.4 Features of Python

- **Easy-to-learn** – Python has few keywords, simple structure, and a clearly defined syntax. This allows the student to pick up the language quickly.
- **Easy-to-read** – Python code is more clearly defined and visible to the eyes.
- **Easy-to-maintain** – Python's source code is fairly easy-to-maintain.
- **A broad standard library** – Python's bulk of the library is very portable and cross-platform compatible on UNIX, Windows, and Macintosh.
- **Interactive Mode** – Python has support for an interactive mode which allows interactive testing and debugging of snippets of code.
- **Portable** – Python can run on a wide variety of hardware platforms and has the same interface on all platforms.
- **Extendable** – You can add low-level modules to the Python interpreter. These modules enable programmers to add to or customize their tools to be more efficient.
- **Databases** – Python provides interfaces to all major commercial databases.
- **GUI Programming** – Python supports GUI applications that can be created and ported to many system calls, libraries and windows systems, such as Windows MFC, Macintosh, and the X Window system of Unix.
- **Scalable** – Python provides a better structure and support for large programs than shell scripting.

Apart from the above-mentioned features, Python has a big list of good features, few are listed below –

- It supports functional and structured programming methods as well as OOP.
- It can be used as a scripting language or can be compiled to byte-code for building large applications.
- It provides very high-level dynamic data types and supports dynamic type checking.
- IT supports automatic garbage collection.
- It can be easily integrated with C, C++, COM, ActiveX, CORBA, and Java.

3.5.5 Libraries used in python:

- Numpy - mainly useful for its N-dimensional array objects.
- Pandas - Python data analysis library, including structures such as data frames.
- Matplotlib - 2D plotting library producing publication quality figures.
- Scikit-learn - the machine learning algorithms used for data analysis and data mining tasks.



Figure 3.5.5.1: NumPy, Pandas, Matplotlib, Scikit-learn

3.6 TECHNIQUE USED OR ALGORITHM USED

3.6.1 EXISTING TECHNIQUE

➤ GAN Algorithm

Generative adversarial networks (GANs) are algorithmic architectures that use two neural networks, pitting one against the other (thus the “adversarial”) in order to generate new, synthetic instances of data that can pass for real data. They are used widely in image generation, video generation and voice generation.

GANs consists of two networks, a Generator $G(x)$, and a Discriminator $D(x)$. They both play an adversarial game where the generator tries to fool the discriminator by generating data similar to those in the training set. The Discriminator tries not to be fooled by identifying fake data from real data.

3.7 IMPLEMENTATION

CODING:

```
from flask import Flask, redirect, url_for, request, render_template, session

import sys

import os

import re

import numpy as np

from werkzeug.utils import secure_filename


# Keras

from tensorflow.keras.applications.imagenet_utils import preprocess_input,
decode_predictions

from tensorflow.keras.models import load_model

from tensorflow.keras.preprocessing import image

from tensorflow.keras.utils import load_img

#from keras.preprocessing.image import load_img

#import matplotlib.pyplot as plt

import matplotlib.image as mpimg


# load model

model_path = 'train_model.h5'
```

```
model = load_model(model_path)

app = Flask(__name__)

# Define the directory where uploaded files will be stored
app.config['UPLOAD_FOLDER'] = os.path.join('upload')
app.config['UPLOAD'] = os.path.join("")

# Allow file uploads of up to 16 megabytes
app.config['MAX_CONTENT_LENGTH'] = 16 * 1024 * 1024

# Ensure that file extensions are limited to images only
app.config['ALLOWED_EXTENSIONS'] = {'png', 'jpg', 'jpeg', 'gif'}

@app.route("/")
def main():
    return render_template('main.html')

@app.route('/login', methods=['GET', 'POST'])
def login():
    return render_template('login.html')

@app.route("/logout")
```

```
def logout():
    return render_template('main.html')

@app.route('/performance')
def performance():
    return render_template('performance.html')

@app.route("/index")
def index():
    return render_template('index.html')

#creates function

def predictions(img_path, model):
    img = load_img(img_path, target_size=(150, 150, 3))

    x = image.img_to_array(img)
    x = x/255
    x = np.expand_dims(x, axis=0)

    print(x)

    y = model.predict(x)
```

```
print("probabilities:", y)

pred = np.argmax(model.predict(x)[0], axis=-1)

print(pred)
```

```
if pred == 0:

    preds = 'YOUNG AGE'

elif pred == 1:

    preds = 'MIDDLE AGE'

else:

    preds = 'OLD AGE'
```

```
return preds, y

print(preds)
```

```
@app.route("/predicted", methods=['POST'])

def predicted():

    # Get the uploaded image file from the form

    uploaded_file = request.files['imagefile']

    # print(uploaded_file)
```

```
# Save the file to a temporary directory

filename = secure_filename(uploaded_file.filename)

# print(filename)


#img_path = os.path.join(app.config['UPLOAD_FOLDER'], filename)

# print(img_path)

img_path = "static/" + filename

uploaded_file.save(img_path)


img_upload = mpimg.imread(img_path)

# print(img_upload)


# Add a third dimension to the image array to indicate that it is a grayscale image

img_upload = np.expand_dims(img_upload, axis=2)

rgb_image = np.repeat(img_upload, 3, axis=2)


# Save the image to the 'static' directory

#img = os.path.join('static/savingimg', filename)

#mpimg.imsave(img, rgb_image)

# Save the grayscale image to the 'static' directory

prediction, y = predictions(img_path, model)
```

```
probabilities_string = np.array2string(y, precision=10, separator=',',  
suppress_small=True)
```

```
# Remove the outer square brackets and split the string by comma
```

```
probabilities_list = probabilities_string.strip("[]").split(',')
```

```
# Convert the strings to float values
```

```
probabilities = [float(prob.strip()) for prob in probabilities_list]
```

```
probabilities_dict = {
```

```
'YOUNG AGE': probabilities[0],
```

```
'MIDDLE AGE': probabilities[1],
```

```
'OLD AGE': probabilities[2]
```

```
}
```

```
highest_value = (max(y[0]))*100
```

```
rounded_number = round(highest_value, 2)
```

```
print(rounded_number)
```

```
# Get the prediction for the uploaded image
```

```
#prediction = predictions(img_path, model)
```

```
# Pass the image file path and prediction result to the template
```



```
return render_template('result.html', prediction=prediction, rgb_image=rgb_image,  
probabilities=rounded_number)
```

```
if __name__ == '__main__':
```

```
    app.run()
```

3.8 TESTING AND VALIDATION

3.8.1 GENERAL

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub-assemblies, assemblies and/or a finished product. It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of test. Each test type addresses a specific testing requirement.

3.8.2 DEVELOPING METHODOLOGIES

The test process is initiated by developing a comprehensive plan to test the general functionality and special features on a variety of platform combinations. Strict quality control procedures are used. The process verifies that the application meets the requirements specified in the system requirements document and is bug free. The following are the considerations used to develop the framework from developing the testing methodologies.

3.8.3 TYPES OF TESTS

3.8.3.1 UNIT TESTING

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program input produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application. It is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

3.8.3.2 FUNCTIONAL TEST

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items:

- Valid Input : identified classes of valid input must be accepted.
- Invalid Input : identified classes of invalid input must be rejected.
- Functions : identified functions must be exercised.
- Output : identified classes of application outputs must be exercised.
- Systems/Procedures : interfacing systems or procedures must be invoked.

3.8.3.3 SYSTEM TEST

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

3.8.3.4 PERFORMANCE TEST

The Performance test ensures that the output be produced within the time limits, and the time taken by the system for compiling, giving response to the users and request being send to the system for to retrieve the results.

3.8.3.5 INTEGRATION TESTING

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects.

The task of the integration test is to check that components or software applications, e.g. components in a software system or – one step up – software applications at the company level – interact without error.

3.8.3.6 ACCEPTANCE TESTING

User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements.

Acceptance testing for Data Synchronization:

- The Acknowledgements will be received by the Sender Node after the Packets are received by the Destination Node
- The Route add operation is done only when there is a Route request in need
- The Status of Nodes information is done automatically in the Cache Updation process

3.8.3.7 BUILD THE TEST PLAN

Any project can be divided into units that can be further performed for detailed processing. Then a testing strategy for each of this unit is carried out. Unit testing helps to identify the possible bugs in the individual component, so the component that has bugs can be identified and can be rectified from errors.

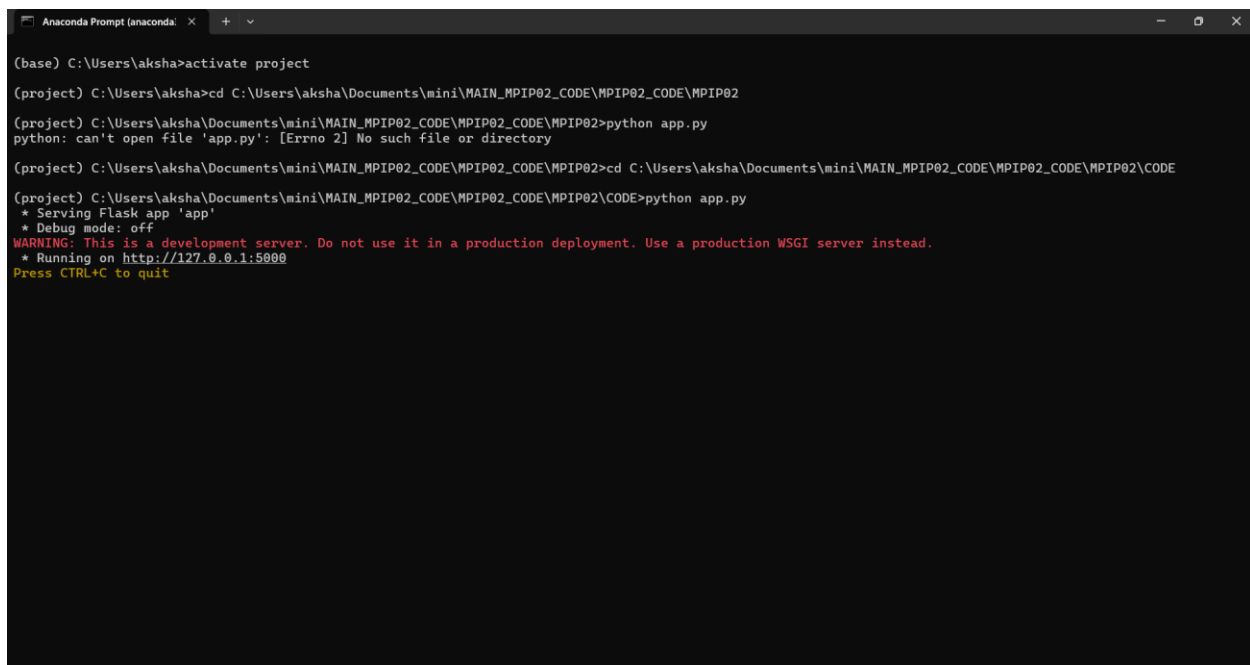
It can be seen in Figure that the optimized reconstructions are closer to the original images than the initial ones. However, the choice is more complicated when it comes to the comparison of the two latent vector optimization approaches.

3.9 SNAPSHOTS

3.9.1 GENERAL

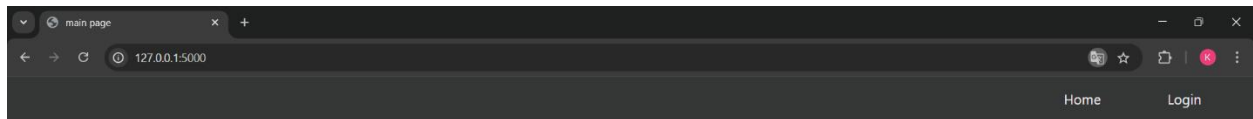
This project implements like application using python and the Server process is maintained using the SOCKET & SERVERSOCKET and the Design part is played by Cascading Style Sheet.

3.9.2 VARIOUS SNAPSHOTS



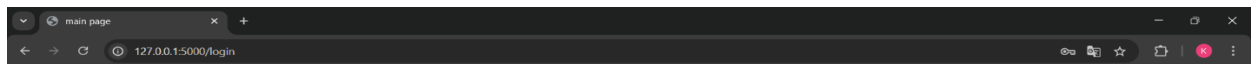
```
(base) C:\Users\aksha>activate project
(project) C:\Users\aksha>cd C:\Users\aksha\Documents\mini\MAIN_MPIP02_CODE\MPIP02_CODE\MPIP02
(project) C:\Users\aksha\Documents\mini\MAIN_MPIP02_CODE\MPIP02_CODE\MPIP02>python app.py
python: can't open file 'app.py': [Errno 2] No such file or directory
(project) C:\Users\aksha\Documents\mini\MAIN_MPIP02_CODE\MPIP02_CODE\MPIP02>cd C:\Users\aksha\Documents\mini\MAIN_MPIP02_CODE\MPIP02_CODE\MPIP02\CODE
(project) C:\Users\aksha\Documents\mini\MAIN_MPIP02_CODE\MPIP02_CODE\MPIP02\CODE>python app.py
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
```

Fig 3.9.2.1: Execution of Code



Facial Age stage Estimation

Fig 3.9.2.2: Home Page



Facial Age Stage Estimation

Login Form

Email address:

Password:

Fig 3.9.2.3: Login Form

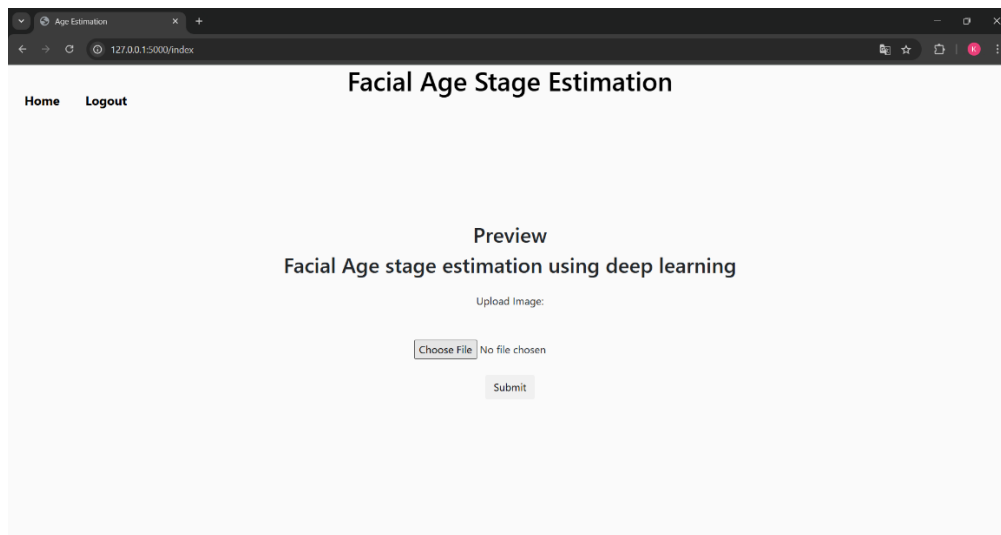


Fig 3.9.2.4: Preview Page

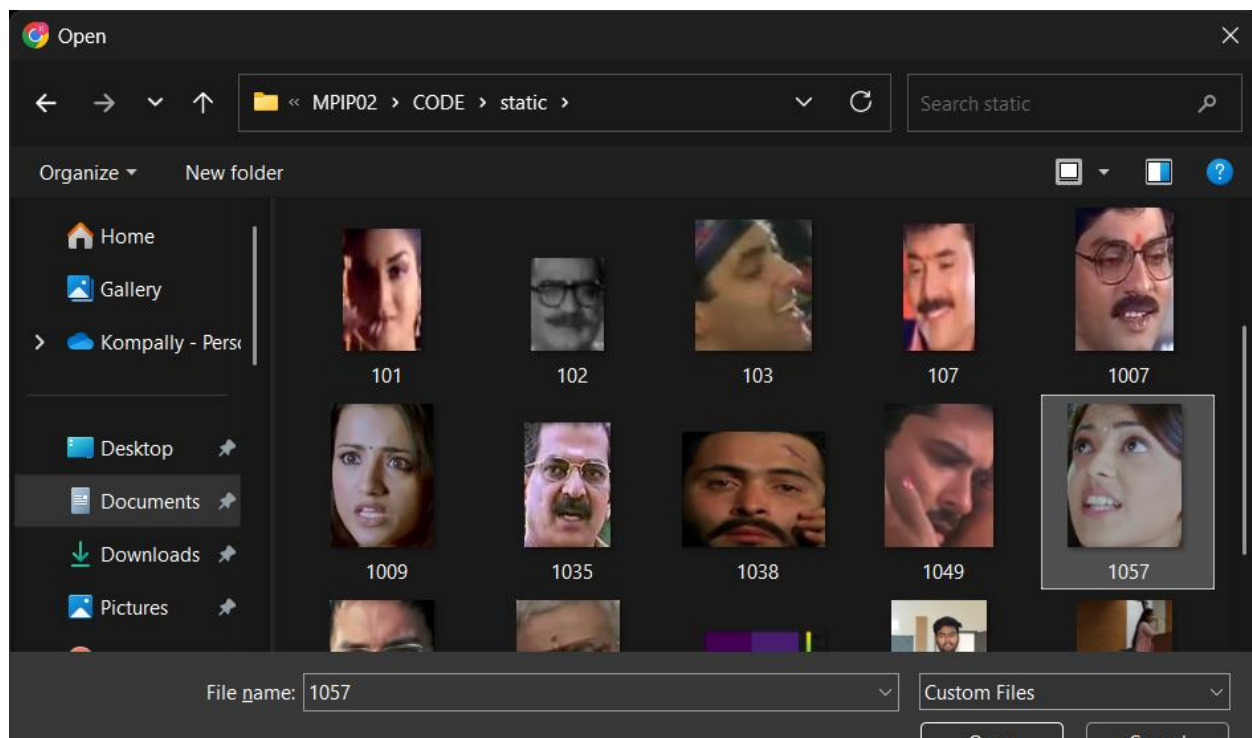


Fig 3.9.2.5 : Input Image

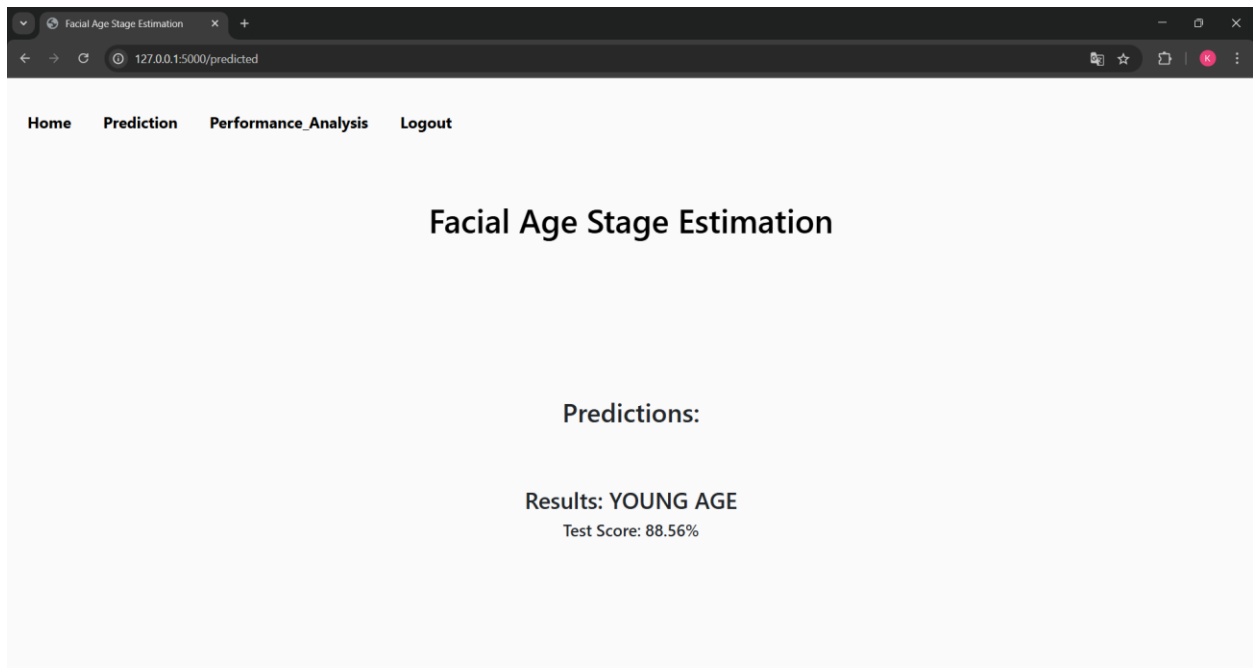


Fig 3.9.2.6: Prediction

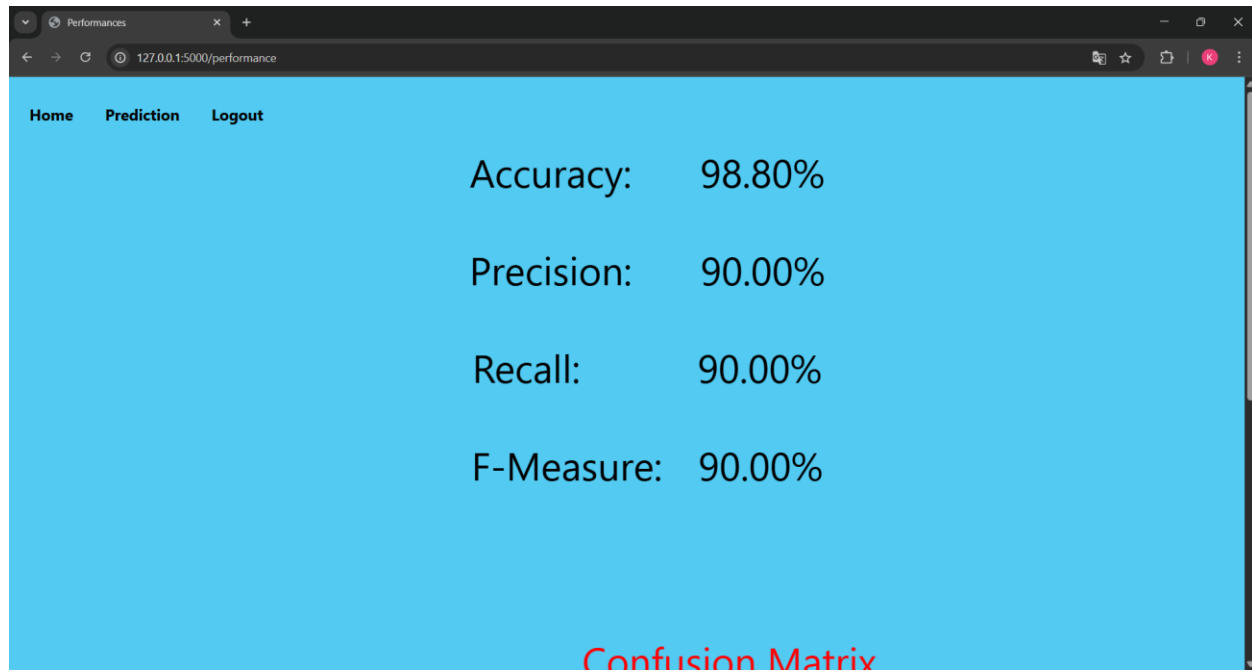


Fig 3.9.2.7 : Performance Analysis

CHAPTER 4

RESULT AND DISCUSSION

4.1 RESULT

The system effectively detected faces in images using pre-trained models like Haar Cascade and MTCNN. After detection, facial regions were preprocessed and passed to the CNN model for age prediction. The model was trained on the UTKFace dataset, which includes a diverse range of facial images labeled with age, gender, and ethnicity.

The system demonstrated good generalization on unseen data and performed reliably across various lighting conditions and facial expressions.

A user-friendly graphical interface was developed using Tkinter, allowing users to upload images or use a webcam to estimate age in real-time. The estimated age was displayed directly on the image, providing an interactive experience.

4.2 FUTURE ENHANCEMENT

It is part of our future work. Moreover, we believe that the face reconstruction part of our method can be further improved by combining “Pixelwise” and “Identity-Preserving” approaches into one optimization objective. We are also planning to explore this path in our further studies.

CHAPTER 5

CONCLUSION

5.1 CONCLUSION

In this work, we have proposed a new effective method for synthetic aging of human faces based on Age Conditional Generative Adversarial Network (Age-cGAN). The method is composed of two steps: (1) input face reconstruction requiring the solution of an optimization problem in order to find an optimal latent approximation z^* , (2) and face aging it-self performed by a simple change of condition y at the input of the generator. The cornerstone of our method is the novel “Identity-Preserving” latent vector optimization approach allowing to preserve the original person’s identity in the reconstruction. This approach is universal meaning that it can be used to preserve identity not only for face aging but also for other face alterations (e.g. adding a beard, sunglasses etc.)

Our face aging method can be used for synthetic augmentation of face datasets and for improving the robustness of face recognition solutions in cross-age scenarios.

5.2 REFERENCES

- [1] Yun Fu, Guodong Guo, and Thomas S Huang, “Age synthesis and estimation via faces: A survey,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 32, no. 11, pp. 1955–1976, 2010.
- [2] Xiangbo Shu, Jinhui Tang, Hanjiang Lai, Luoqi Liu, and Shuicheng Yan, “Personalized age progression with ag-ing dictionary,” in *Proceedings of International Conference on Computer Vision*, Santiago, Chile, 2015.
- [3] Unsang Park, Yiyang Tong, and Anil K Jain, “Age-invariant face recognition,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 32, no. 5, pp. 947–954, 2010.
- [4] Wei Wang, Zhen Cui, Yan Yan, Jiashi Feng, Shuicheng Yan, Xiangbo Shu, and Nicu Sebe, “Recurrent face ag-ing,” in *Proceedings of Computer Vision and Pattern Recognition*, Las Vegas, USA, 2016.
- [5] Bernard Tiddeman, Michael Burt, and David Perrett, “Prototyping and transforming facial textures for perception research,” *IEEE Computer Graphics and Applications*, vol. 21, no. 5, pp. 42–50, 2001.
- [6] Ira Kemelmacher-Shlizerman, Supasorn Suwajanakorn, and Steven M Seitz, “Illumination-aware age progression,” in *Proceedings of Computer Vision and Pattern Recognition*, Columbus, USA, 2014.
- [7] Jinli Suo, Song-Chun Zhu, Shiguang Shan, and Xilin Chen, “A compositional and dynamic model for face aging,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 32, no. 3, pp. 385–401, 2010.
- [8] Yusuke Tazoe, Hiroaki Gohara, Akinobu Maejima, and Shigeo Morishima, “Facial aging simulator considering geometry and patch-tiled texture,” in *Proceedings of ACM SIGGRAPH*, Los Angeles, USA, 2012.

- [9] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio, “Generative adversarial nets,” in Proceedings of advances in Neural Information Processing Systems, Montreal, Canada, 2015.
- [10] Diederik P Kingma and Max Welling, “Auto-encoding variational bayes,” in Proceedings of International Conference on Learning Representations, Banff, Canada, 2014.
- [11] Anders Boesen Lindbo Larsen, Søren Kaae Sønderby, Hugo Larochelle, and Ole Winther, “Autoencoding beyond pixels using a learned similarity metric,” in Proceedings of International Conference on Machine Learning, New York, USA, 2016.
- [12] Guim Perarnau, Joost van de Weijer, Bogdan Raducanu, and Jose M Alvarez, “Invertible conditional gans for image editing,” in Proceedings of advances in Neural Information Processing Systems Workshops, Barcelona, Spain, 2016.
- [13] Mehdi Mirza and Simon Osindero, “Conditional generative adversarial nets,” in Proceedings of advances in Neural Information Processing Systems, Montreal, Canada, 2014.
- [14] Jon Gauthier, “Conditional generative adversarial nets for convolutional face generation,” Class Project for Stanford CS231N: Convolutional Neural Networks for Visual Recognition, 2014.
- [15] Alec Radford, Luke Metz, and Soumith Chintala, “Un-supervised representation learning with deep convolutional generative adversarial networks,” in Proceedings of International Conference on Learning Representations, San Juan, Puerto Rico, 2016.
- [16] Diederik Kingma and Jimmy Ba, “Adam: A method for stochastic optimization,” arXiv preprint arXiv:1412.6980, 2014.
- [17] Jun-Yan Zhu, Philipp Krähenbühl, Eli Shechtman, and Alexei A Efros, “Generative visual manipulation on the natural image manifold,” in Proceedings of European Conference on Computer Vision, Amsterdam, Netherlands, 2016.
- [18] Florian Schroff, Dmitry Kalenichenko, and James Philbin, “Facenet: A unified embedding for face recognition and clustering,” in Proceedings of Computer Vision and Pattern Recognition, Boston, USA, 2015.

- [19] Richard H Byrd, Peihuang Lu, Jorge Nocedal, and Ciyou Zhu, “A limited memory algorithm for bound constrained optimization,” *SIAM Journal on Scientific Computing*, vol. 16, no. 5, pp. 1190–1208, 1995.
- [20] Grigory Antipov, Moez Baccouche, Sid-Ahmed Berrani, and Jean-Luc Dugelay, “Apparent age estimation from face images combining general and children-specialized deep learning models,” in *Proceedings of Computer Vision and Pattern Recognition Workshops*, Las Vegas, USA, 2016.
- [21] Rasmus Rothe, Radu Timofte, and Luc Van Gool, “Dex: Deep expectation of apparent age from a single image,” in *Proceedings of International Conference on Computer Vision Workshops*, Santiago, Chile, 2015.
- [22] Brandon Amos, Bartosz Ludwiczuk, and Mahadev Satyanarayanan, “Openface: A general-purpose face recognition library with mobile applications,” *Tech. Rep.*, CMU-CS-16-118, CMU School of Computer Science, 2016.